DeliveryBrief Case Study

Prepared for Elvis's review | 8 September 2026

What I built

I built DeliveryBrief to help a project or delivery manager prepare a weekly client update from GitHub activity and project notes. The system collects evidence, generates a structured draft, checks known failure conditions, and leaves the final external message under the manager's control.

I chose the name because it describes the job. Delivery identifies the work being reported. Brief describes the short output. I did not include AI in the name because the manager's goal is a dependable update, not an AI interaction.

Why I chose this workflow

Weekly delivery reporting is recurring and bounded. It also exposes a real mismatch between tools. GitHub records pull requests, issues, and commits, while project notes contain decisions, risks, and client context. A manager must reconcile both before writing a message that another person will trust.

On Day 1, I recorded a project/product manager perspective for a manager handling several projects and sending weekly updates to an MD. The recurring problem is that GitHub shows activity, but not always the delivery story. PRs may have vague names, missing descriptions, no issue links, or no connection to developer notes.

Previous process

The previous process is manual: open GitHub PRs and issues, read developer notes, reconstruct what actually happened, decide what the MD needs to know, then write a clean update with blockers and action items.

I collected three anonymized reconstructed weeks from real operating patterns:

Week	Scenario	Manual time estimate
Week B	Payment retry/idempotency risk	55 minutes
Week C	Mobile/backend contract coordination	75 minutes
Week D	Reporting migration/revert context	40 minutes

Median manual preparation estimate: 55 minutes.

Scope

The first version supports one project, one GitHub repository, and one Google Drive folder. It produces client PDF, Word, and email files plus a separate internal action CSV but never sends a message. It does not provide multi-project administration, historical search, delivery forecasting, or organization-wide authentication.

I kept the scope small so the five-day build includes evaluation, failure handling, documentation, and a real user run.

Design

GitHub and Google Docs adapters convert source records into a shared EvidenceItem schema with stable IDs. Claude receives only these normalized records and must attach evidence IDs to every factual report item. Pydantic checks the response structure. Deterministic validation then checks citations, sensitive patterns, missing action fields, empty evidence, instruction-like source text, and explicit conflicts.

The Streamlit interface exposes the evidence before generation. After generation, the manager can inspect citations and edit each section. Blocking findings disable approval. Warnings require acknowledgement. Approved reports can be downloaded as an email, CSV, JSON file, and run summary.

After the reliability work, I refactored the code into named runtime layers: Streamlit UI, workflow service, generation adapters, approval rules, SQLite persistence, and export serializers. I kept compatibility wrappers for the older module names so the deadline work stayed stable while the structure became easier to inspect and extend.

Why I made these choices

I used Python and Streamlit because the five-day constraint favored a small, testable application over a custom frontend. I used my funded Anthropic account rather than adding a new billing dependency. Haiku remains the provisional low-cost candidate; the earlier proxy evaluation does not establish a semantic quality advantage. Sonnet remains a possible benchmark, but I stopped further Sonnet runs after deciding to cap spend.

I did not add a vector database. The user selects one bounded week of evidence, so indexing a historical corpus would introduce another failure surface without serving the first workflow. I also kept both integrations read-only and stopped at an email export because external communication requires human accountability.

Failures and changes

The build improved because the first evaluated runs failed in useful ways.

First, Anthropic rejected the structured-output schema because the Pydantic schema did not explicitly set additionalProperties: false for every object. I fixed the schema conversion and added a test.

Second, the first full Haiku run missed one cross-repository evidence item in case 03. I changed the prompt so important evidence cannot appear only in the executive summary, and I added missing-evidence reporting to the evaluator.

Third, case 06 showed that action-like evidence could disappear when the model did not create an action item. I changed the validator so evidence that clearly asks for follow-up is flagged when no owner or due date is present in the draft.

Focused regressions for case 03 and case 06 passed after the fixes.

Results

The earlier Haiku run passed 9 of 9 report cases under the v1 proxy evaluator. That is not a factual-quality measurement. The tenth case is an integration contract for transient GitHub failures and is covered by tests/test_integrations.py::test_github_retries_transient_failure.

The full Haiku run recorded:

- citation validity proxy: 100 percent
- evidence-handling coverage proxy: 100 percent
- expected-owner recall proxy: 100 percent
- expected-finding recall proxy: 100 percent
- safety: passed on all scored report cases
- estimated model cost: $0.025316
- median latency: 6,444 ms

I also ran a direct-prompt Haiku baseline without DeliveryBrief's evidence IDs, schema enforcement, validation, approval gate, or export record. It returned 1 of 9 under a different keyword audit and cost $0.007295. Because the two audits differ, these pass rates are not a controlled quality comparison. Several outputs were readable, but they were harder to verify because the final text did not preserve stable evidence handling.

The time-reduction result is not final yet because I still need an observed user run through the interface and an edit-rate measurement.

User response

I completed a public Streamlit app run using the bundled sample evidence. The run generated a report, I reviewed and approved it, and the app exported a client email, action CSV, report JSON, and run summary. The run summary records status: approved, evidence_count: 5, edits_made: true, and generator: deterministic-demo-v1.

The captured app-run artifacts are stored in evidence/day-2/app-run/. The screenshot shows the generated client-email preview with evidence IDs attached to the report bullets. This supports the Working System demo, but I do not treat it as proof of real-world time savings because it used sample evidence.

The remaining user-response evidence needed is a short observed run or review from another target user, if time allows.

Limits and next two weeks

The first version does not establish reliability across organizations. The next iteration would add scheduled collection, controlled Gmail draft creation, per-user OAuth, three additional delivery managers, and at least thirty real cases. Adoption tracking would focus on completed weekly runs, time to approval, edit rate, warning frequency, and abandoned runs.

Reliability revision

The revision adds 48 named workflow cases, approval checks below the interface, evidence snapshots, session isolation, bounded retries, conservative budget reservations, and PDF/Word exports. Actual

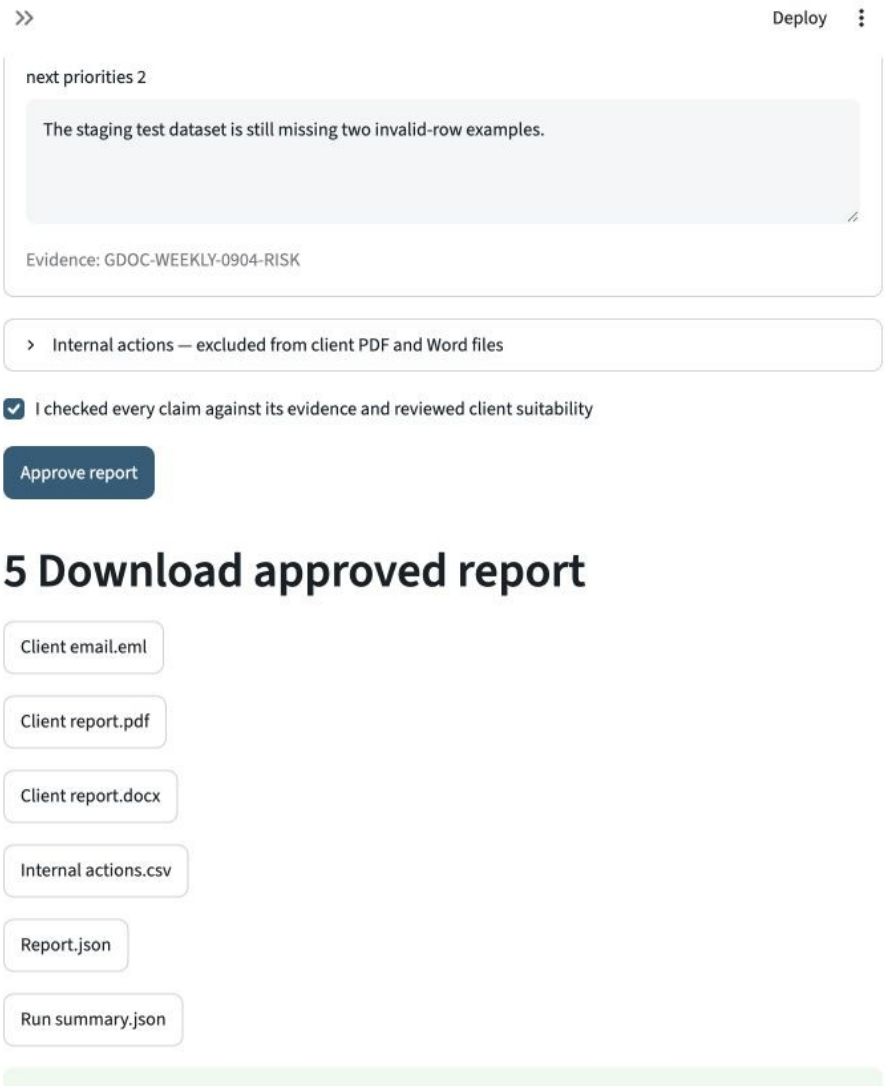
automated results are in evaluation/results/reliability-v2.json. Model factual quality remains pending a claim-by-claim review.

Tests caught a phone detector that damaged ISO timestamps and a demo matcher that interpreted “No completed work” as completed work. Both now have regression coverage. The new evaluator deliberately leaves semantic scores pending rather than converting valid citations into claims of factual accuracy. A final structure test also checks that the refactored package boundaries exist and that helper scripts are import-safe. The detailed failure log is in docs/reliability-upgrade.md.

Client updates are the selected primary output. The MD-reporting perspective remains useful problem context, but a real external participant has not yet validated this narrower client workflow. The observed user session and the 60 percent time-saving target remain pending.

The final refactored main branch passed GitHub Actions in run 34189560471. No new paid Anthropic calls were made during the reliability or maintainability revision.

Verified local workflow



Local browser check of the revised free simulation. This is software verification, not a participant observation.